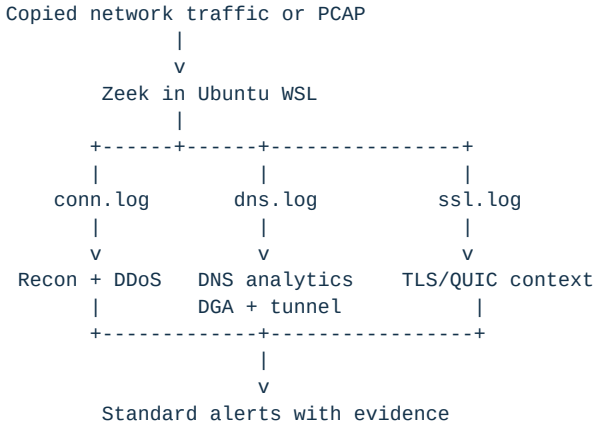
AEGISFLOW

Passive AI-Assisted Cyber-Threat Detection

REPORT	SIH Engineering Build Walkthrough
COVERAGE	Sprints 0 to 5
STATUS	51 automated tests passing
SNAPSHOT	30 August 2026

1. What we are building

AegisFlow observes copied network traffic on the monitoring side of a one-way or passively monitored network. It does not send packets back toward the protected network. Zeek converts packet captures into structured connection, DNS, and encrypted-session metadata. AegisFlow then applies transparent statistical detectors and focused machine-learning models to produce evidence-rich alerts.



2. Current progress at a glance

Sprint	Main outcome	Status	Demonstration result
Sprint 0	Shared contracts, Zeek connection ingestion, scan detection	Complete	Vertical and horizontal scan alerts
Sprint 1	WSL Zeek bridge, real PCAP processing, SYN/UDP flood detection	Complete	Real PCAP: 12 connections processed
Sprint 2	DNS ingestion, DGA model, DNS-tunnelling detector	Prototype complete	Threat fixture: 21 events, 2 alerts
Sprint 3	C2 beacon detection and TLS/QUIC metadata context	Prototype complete	Beacon fixture: 8 events, 1 alert
Sprint 4	Exfiltration detection, incident correlation, scoring and feedback	Prototype complete	C2 + exfiltration: 2 alerts, 1 critical incident
Sprint 5	Persistent analyst API, dashboard, Docker, and PostgreSQL	Complete	Healthy dashboard; restart-safe incident storage

Production dataset acquisition and deployment-specific calibration remain future work. Smoke-test metrics are never presented as operational accuracy.

3. Environment and the Windows/Linux solution

Zeek is designed for Linux. The development computer runs Windows, so Ubuntu WSL 2 is used as the Linux runtime. Zeek 8.0.10 is installed at `/opt/zeek/bin/zeek`. AegisFlow itself runs on Windows and calls Zeek through a tested WSL bridge.

The bridge performs these operations automatically:

1. Confirm that WSL and Zeek are available. 2. Convert Windows paths such as `C:\project\capture.pcap` to WSL paths such as `/mnt/c/project/capture.pcap`. 3. Run Zeek inside Ubuntu. 4. Write logs back into the Windows project directory. 5. Feed the generated JSON logs to AegisFlow detectors.

The real sample `/home/shukl/zeek-test/sample.pcap` completed this full path. Zeek produced `conn.log`, `dns.log`, `ssl.log`, and capture-filter telemetry.

4. Sprint 0 walkthrough - foundation and reconnaissance

Goal

Prove the smallest complete path from a Zeek connection record to an explainable security alert.

What was built

- Shared `NetworkEvent`, `Evidence`, and `Alert` contracts.
- Zeek `conn.log` JSON reader with line-numbered validation errors.
- Sliding observation windows grouped by source device.
- Vertical port-scan detection: one source probes many ports on one host.
- Horizontal host-scan detection: one source probes the same port across many hosts.
- Cooldown and deduplication to reduce repeated alerts.
- JSON alert output containing confidence, severity, flow identifiers, evidence,

thresholds, and limitations.

Why it matters

This sprint established the contract used by later detectors. The dashboard can eventually display alerts from different attacks without threat-specific UI code.

5. Sprint 1 walkthrough - PCAP, WSL, health, and DDoS

Goal

Accept a raw capture through Zeek and identify high-rate connection behaviour.

What was built

- Windows-to-WSL Zeek execution bridge.
- PCAP-to-Zeek conversion command.
- Reusable keyed sliding-window engine with bounded out-of-order support.
- SYN-flood detection using incomplete connection attempts and their ratio.
- UDP-flood detection using packet and byte volume.
- Replay-health report with event count, event span, ordering, and processing rate.
- Dataset manifest and leakage-safe split policy for CICDDoS2019.

Real-machine result

The real sample capture produced 12 normalized connection events over approximately 120 seconds. No attack alert was raised because the sample did not cross the scan or DDoS thresholds. Zero alerts on ordinary traffic is a valid result.

6. Sprint 2 walkthrough - DNS analytics and first trained model

Goal

Detect two different DNS threats without treating every attack as the same ML task.

DNS-tunnelling path

The detector groups queries by client and approximate base domain inside a sliding window. It measures:

- query count;
- unique subdomain labels;
- average subdomain length;
- average character entropy.

An alert requires high volume and diversity plus long or high-entropy labels. CDN, endpoint-security, hosted telemetry, and allowlist cases are documented because they can resemble tunnelling.

DGA model training

The first model is an inspectable character 3-gram Multinomial Naive Bayes model. For example, `google.com` is converted into overlapping groups such as `^go`, `goo`, `oog`, and `ogl`. Training learns which groups are more frequent in benign or synthetic DGA-like domains.

Training produced three artefacts:

1. A JSON model containing learned n-gram counts and class totals. 2. A JSON evaluation report with confusion-matrix metrics. 3. A model card describing purpose, data, results, risks, and limitations.

The demonstration split contains 20 training examples and 12 test examples. Duplicate domains cannot appear across splits, and malicious families cannot be shared between train and test. The tiny holdout scored 12 of 12 correctly. This only verifies the pipeline; it is not a real-world accuracy claim.

Demonstration result

- Synthetic DNS threat input: 21 DNS events.
- Output: one DGA-like alert and one DNS-tunnelling alert.
- Real sample DNS log: 7 events and zero alerts.
- Encrypted DNS remains invisible unless telemetry is collected before encryption.

7. Sprint 3 walkthrough - C2 beacon and encrypted metadata

Goal

Detect repeated C2-like callbacks without decrypting TLS or QUIC payloads.

Beacon detection logic

Connections are grouped by source, destination, destination port, and protocol. Once enough repeated connections exist, the detector measures:

- mean time between connections;
- timing coefficient of variation, representing jitter;
- total-byte coefficient of variation;
- number of repeated connections in the window.

A beacon candidate requires a plausible callback interval, low timing variation, and stable transfer sizes. A rare fingerprint cannot create an alert by itself.

TLS and QUIC context

The encrypted-session adapter reads metadata such as:

- TLS or QUIC version;
- cipher;
- visible server name;
- application protocol;
- client and server fingerprints when Zeek provides them;
- handshake establishment and resumption state.

A contextual anomaly score can slightly adjust beacon confidence. The alert still requires timing and size evidence. Payloads are never decrypted.

False-positive handling

- Irregular scheduled traffic is tested and does not alert.
- Highly variable transfer sizes do not alert.
- Approved destination IPs can be allowlisted.
- Software updates, monitoring checks, NAT, packet loss, and observation gaps are

listed as limitations in every alert.

Demonstration result

- Synthetic beacon input: 8 connection events and 8 TLS metadata records.
- Output: one periodic-beacon alert.
- Mean interval: 10.02 seconds.

- Interval variation: 0.0147.
- Size variation: 0.0.
- Real sample: 12 connections, 2 TLS records, and zero C2 alerts.

8. Sprint 4 walkthrough - exfiltration and incidents

Goal

Detect suspicious outbound transfer behaviour and combine related detector alerts into one inspectable incident without treating ordinary backups as attacks.

Exfiltration detection logic

The detector maintains a small per-source history and groups flows by source and destination inside a sliding window. It measures:

- total outbound bytes;
- outbound-to-inbound byte ratio;
- average outbound bytes per flow;
- comparison against the source's prior median flow size;
- destination age as supporting context.

An alert requires large outbound volume, a strongly outbound direction, and a clear increase over the source baseline. Destination novelty is evidence, not a sole trigger. Approved backup destinations are excluded through an explicit allowlist.

Incident correlation and scoring

Alerts from the same source inside a configurable time window are correlated into a single incident. Every incident retains all contributing alert IDs, detector IDs, threat types, first and last timestamps, and deterministic scoring factors.

Risk score version 1 contains three inspectable components:

1. Fixed weights for each distinct threat category. 2. A cross-detector bonus for a multi-stage story. 3. A bounded confidence contribution worth at most 20 points.

Confidence and severity remain separate. Replaying the same alert ID is idempotent and does not duplicate an incident contribution.

Analyst feedback

Validated feedback records support `confirmed_malicious`, `benign`, and `needs_review` dispositions. Each record includes an incident ID, analyst label, timestamp, notes, and deterministic feedback ID. The in-memory prototype defines the contract that the Sprint 5 database and API will persist.

Demonstration result

- Synthetic exfiltration input: 8 connection events.
- Output: one outbound-volume anomaly alert.
- Observed outbound bytes: 1,530,000.

- Outbound-to-inbound ratio: 510.0.
- Source baseline median: 11,000 bytes per flow.
- C2 plus exfiltration correlation: 2 alerts became 1 critical incident.
- Incident risk score: 100 with all scoring components retained.
- Approved backup, balanced download, deduplication, and feedback validation tests pass.

9. Sprint 5 walkthrough - analyst API and dashboard

Goal

Turn detector output into a practical investigation workflow that an analyst can use without sending any traffic back toward the protected network.

Persistent analyst service

The FastAPI service now provides versioned endpoints for:

- service health and operational mode;
- risk-prioritized incident queues with status and severity filters;
- incident detail with every contributing alert, evidence item, and limitation;
- incident status updates;
- validated analyst feedback;
- portable JSON incident export;
- idempotent demonstration-data import.

The local mode uses SQLite so the demo starts with minimal infrastructure. The repository creates indexes for the queue and investigation views and closes every database connection deterministically. A PostgreSQL adapter, explicit schema, and Docker Compose topology are included for the durable deployment path.

Analyst console

The React and TypeScript console uses a compact security-operations layout. Its first view shows active, critical, average-risk, and reviewed metrics followed by a searchable incident queue. Selecting an incident opens the evidence drawer in one interaction. The drawer keeps three concepts visibly separate:

- Risk score - deterministic policy priority from 0 to 100.
- Severity - policy category used for triage.
- Confidence - detector certainty, which is not operational impact.

The analyst can inspect the attack timeline, source-to-destination route, raw evidence explanations, score components, and earlier feedback. They can then change workflow status, record a malicious/review/benign disposition, or download the full incident as JSON.

Offline and one-way operation

The production frontend is compiled into static assets and served by the same API process. No external font, image, analytics, or cloud service is required at runtime. The Docker bundle contains the API, dashboard, and PostgreSQL database. Every component remains on the monitoring side and the API health response explicitly reports that a return path is not required.

Demonstration result

- Production dashboard build completed successfully.
- Local API health returned `healthy` in `passive_offline` mode.
- Demonstration import persisted 1 incident, 2 alerts, and 1 feedback record.
- The incident queue returned the critical multi-stage incident at risk score 100.
- Queue, detail, status, feedback, error, and export workflows passed API tests.
- The complete suite now contains 51 passing automated tests.
- Docker Desktop 4.88.1, Engine 29.7.2, and Compose 5.4.0 were verified.
- PostgreSQL and analyst-console containers started successfully and stayed healthy.
- The containerized API reported `postgresql` storage, imported 1 incident, 2 alerts,

and 1 feedback record, and returned the risk-100 incident.

- The containerized dashboard returned HTTP 200 at `127.0.0.1:8000`.
- The risk-100 incident remained available after separate API and PostgreSQL

container restarts, confirming volume-backed persistence.

10. Limitation fix 1 - automatic incident persistence

Problem

Previously, detectors wrote alerts to JSON files and correlation produced an incident file, but the analyst database needed a separate import or **Load demo data** action. That gap was acceptable for a staged demo but not for a continuous monitoring path.

What changed

- `correlate-alerts` now reads `--database` or `AEGISFLOW_DB`.
- After correlation, alerts and incidents are upserted into SQLite or PostgreSQL.
- Replaying the same alert IDs is idempotent, so it does not create duplicates.
- Detector reprocessing updates machine-produced evidence but does not overwrite the

analyst-owned incident status.

- The command report records whether persistence ran, which backend was used, and the

number of alerts and incidents persisted.

End-to-end proof

The rebuilt Docker deployment processed two fresh alerts for source `10.0.0.77`: one periodic C2 alert and one outbound-exfiltration alert. Correlation automatically created incident `9c4d8ea38d6860645e61` in PostgreSQL with risk score 100 and critical severity. It appeared through the dashboard API without calling `/api/demo/load`.

The incident was then marked `investigating` and the same alert pair was processed again. PostgreSQL still contained two total incidents, the new incident still had two alerts, and its status remained `investigating`. This proves both idempotency and safe preservation of analyst work.

11. What is rule-based and what is machine learning

Threat	Current method	Why
Vertical/horizontal scans	Statistical thresholds	Fan-out behaviour is direct and explainable
SYN/UDP floods	Statistical windows	Rate, volume, and completion evidence are primary signals
DGA-like domains	Trained character n-gram model	Domain character patterns benefit from supervised learning
DNS tunnelling	Behavioural window	Repetition and subdomain diversity matter more than one name
C2 beaconing	Timing and size statistics	Periodicity can be measured without payload access
TLS/QUIC fingerprint	Supporting metadata only	Fingerprints are not reliable proof of malware by themselves
Outbound exfiltration	Statistical baseline and directional ratios	Volume alone is insufficient without direction and source history
Incident scoring	Deterministic policy	Analysts must be able to reproduce every risk-score component

A separate ML model is therefore not trained for every attack. Models are introduced only when labelled data and learned patterns add value beyond transparent rules.

12. Verification summary

The current suite contains 51 automated tests. It covers:

- valid and malformed Zeek connection and DNS records;
- Windows-to-WSL path translation and Zeek invocation;
- scan, SYN-flood, UDP-flood, DGA, DNS-tunnel, and C2 alerts;
- window expiry and out-of-order events;
- benign contacts, hosted-service domains, scheduled traffic, variable transfers,

allowlists, and fingerprint-only suppression;

- dataset leakage checks and model probability bounds.
- exfiltration baselines, outbound ratios, approved backups, correlation,

deduplication, deterministic scoring, and analyst feedback.

- database idempotency, persistence, filtering, workflow validation, API health,

queue-to-detail navigation, review, error responses, and incident export.

- automatic SQLite/PostgreSQL persistence from correlation and analyst-status

preservation during reprocessing.

13. Current limitations

- DGA metrics use a tiny synthetic dataset; production data is not yet acquired.
- The base-domain function uses a two-label approximation until a reviewed public

suffix snapshot is included.

- Current thresholds require environment-specific calibration.
- Encrypted payloads and ordinary DNS-over-HTTPS contents are not inspected.
- NAT can combine multiple devices behind one source address.
- Alert correlation within one command run is still in memory, but completed incidents

and their alerts are now persisted automatically to SQLite or PostgreSQL.

- Detector baselines and unfinished streaming correlation windows do not yet survive a

detector-process restart.

- Incident weights are transparent initial policy values, not calibrated operational risk.
- Authentication and role-based permissions are not included in the SIH prototype.
- Authentication and secret management must be strengthened before using the Docker

topology outside an isolated demonstration environment.

14. Next sprint

Sprint 6 will replace remaining assumptions with reproducible per-threat metrics, latency and throughput benchmarks, loss/skew/malformed-input testing, restart recovery, and a formal threat model.

Appendix A - Repeatable commands

From the `aegisflow` directory, set `PYTHONPATH=src` for the current shell.

Test everything

```
python -m unittest discover -s tests -v
```

Verify Zeek in WSL

```
python -m aegisflow.cli check-zeek
```

Train the DNS demonstration model

```
python -m aegisflow.cli train-dns `
  --dataset examples/dns_training_demo.csv `
  --model-output output/models/dns_dga_demo.json `
  --metrics-output output/dns_model_metrics.json `
  --model-card-output output/DNS_MODEL_CARD.md
```

Replay DNS threats

```
python -m aegisflow.cli dns-replay `
  --input examples/zeek_dns_threats.jsonl `
  --model output/models/dns_dga_demo.json `
  --output output/sprint2_dns_alerts.jsonl `
  --report-output output/sprint2_dns_report.json
```

Replay C2 beacon traffic with TLS context

```
python -m aegisflow.cli c2-replay `
  --input examples/zeek_conn_beacon.jsonl `
  --encrypted-input examples/zeek_ssl_beacon.jsonl `
  --output output/sprint3_c2_alerts.jsonl `
  --report-output output/sprint3_c2_report.json
```

Replay exfiltration behaviour

```
python -m aegisflow.cli exfil-replay `
  --input examples/zeek_conn_exfil.jsonl `
  --output output/sprint4_exfil_alerts.jsonl `
  --report-output output/sprint4_exfil_report.json
```

Correlate C2 and exfiltration alerts

```
python -m aegisflow.cli correlate-alerts `
  --input output/sprint3_c2_alerts.jsonl `
  --input output/sprint4_exfil_alerts.jsonl `
  --output output/sprint4_incidents.jsonl `
  --report-output output/sprint4_incident_report.json
```

To persist automatically, set the repository before running the same command:

```
$env:AEGISFLOW_DB = "output/aegisflow.db"
```

Build and run the analyst console

```
python -m pip install -e ".[api]"
cd web
pnpm install
pnpm run build
cd ..
$env:AEGISFLOW_ROOT = (Get-Location).Path
$env:AEGISFLOW_WEB = (Join-Path (Get-Location).Path "web/dist")
python -m uvicorn aegisflow.api:app --host 127.0.0.1 --port 8000
```

Open `http://127.0.0.1:8000`. The first empty state provides a **Load demo data** button. On a Docker-enabled system, the PostgreSQL bundle starts with:

```
docker compose up --build
```